ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/317230130

Heterogeneous computing for a hybridizable discontinuous Galerkin geometric multigrid method

Article · May 2017

CITATION
1

READS
191

3 authors, including:



Matthew Knepley
University at Buffalo, The State University of New York
170 PUBLICATIONS **2,968** CITATIONS

SEE PROFILE

Some of the authors of this publication are also working on these related projects:

Computational Geophysics View project

PDE Solvers View project

All content following this page was uploaded by Matthew Knepley on 21 July 2017.

The user has requested enhancement of the downloaded file.

arXiv:1705.09907v2 [math.NA] 4 Jun 2017

HETEROGENEOUS COMPUTING FOR A HYBRIDIZABLE DISCONTINUOUS GALERKIN GEOMETRIC MULTIGRID METHOD

MAURICE S. FABIEN*, MATTHEW G. KNEPLEY* , RICHARD T. MILLS , AND BÉATRICE M. RIVIERÈ*

Abstract. We present a heterogeneous computing strategy for a hybridizable discontinuous Galerkin (HDG) geometric multigrid (GMG) solver. Parallel GMG solvers require a combination of coarse grain and fine grain parallelism is utilized to improve time to solution performance. In this work we focus on fine grain parallelism. We use Intel's second generation Xeon Phi (Knights Landing) to enable acceleration. The GMG method achieves ideal convergence rates of 0.2 or less, for high polynomial orders. A matrix free (assembly free) technique is exploited to save considerable memory usage and increase arithmetic intensity. HDG enables static condensation, and due to the discontinuous nature of the discretization, we developed a matrix-vector multiply routine that does not require any costly synchronizations or barriers. Our algorithm is able to attain 80% of peak bandwidth performance for higher order polynomials. This is possible due to the data locality inherent in the HDG method. Very high performance is realized for high order schemes, due to good arithmetic intensity, which declines as the order is reduced.

Key words. finite elements, discontinuous Galerkin, multigrid, high performance computing

AMS subject classifications. 68Q25, 68R10, 68U05

1. Introduction. Multigrid methods are among the most efficient solvers for linear systems that arise from the discretization of partial differential equations. The effectiveness of this multilevel technique was first observed by Fedorenko ([15]) in 1964, and popularized by Brandt ([9]) in 1977. Traditionally, multigrid methods have been applied to low order finite difference, finite volume, and continuous finite element approximations discretizations ([36], [39], [17], [10]). The discontinuous Galerkin (DG) discretization brings with it many advantages: can handle complex geometries, has access to *hp*-adaptivity, is capable of satisfying local mass balance (ideal for flow problems and hyperbolic PDEs), and highly parallelizable due to the lack of continuity constraints between elements. However, DG methods often have more degrees of freedom than their continuous counterpart. In addition, high order discretizations r apidly increase the condition number of the discretization operator, which poses a challenge for any linear solver ([6]).

Originally multigrid methods were developed at a time where access to parallelism was limited, and iterative methods that had less concurrency but better convergence rates with were favored. The multilevel nature of multigrid can cause challenges in its parallelization; load balancing problems occur due to fine grids have ample data to work with, but coarser levels do not. Moreover, traditional multigrid is a multiplicative method, that is, each level must be completely processed before moving to the next. Additive multigrid methods allow for the simultaneous processing of all levels, but the trade off between concurrency and robustness is often not ideal ([7]).

Multiplicative multigrid is well known to have sequential complexity, for N data points a single cycle costs $\mathcal{O}(N)$ floating point operations. However, the parallel complexities of V and F cycles are polylogarithmic. For multiplicative multigrid, a natural heterogeneous computing strategy is to process fine levels up to a threshold on an accelerator, and have the remaining coarse levels be processed by coarse grain parallelism. In this paper we explore a single node solution, say one CPU and one accelerator.

*Department of Computational and Applied Mathematics, Rice University, Houston, TX.

2. Model problem. Consider the elliptic problem

$$-\nabla \cdot (\mathbf{K}\nabla u) = f, \quad \text{in} \quad \Omega, \tag{1}$$

$$u = g_D, \quad \text{on} \quad \partial\Omega. \tag{2}$$

where Ω is an open domain in $\mathbb{R}^2$ and $\partial\Omega$ denotes the boundary of the domain. Dirichlet datum g_D is imposed on the boundary. The vector $\mathbf{n}$ is the outward normal vector to Ω. The function f is the prescribed source function and the matrix $\mathbf{K}$ is symmetric positive definite with piecewise constant entries. The div-grad operator in (1) appears in several problems in engineering such as multiphase flow in porous media. It will provide insight into how a DG multigrid solver performs on modern architectures. Moreover, it acts as a gateway to construct very efficient numerical methods for time dependent PDEs that require pressure solves or implicit time stepping.

3. Discretization. HDG methods were designed to address the concern that DG schemes generate more degrees of freedoms (DOFS) when compared to continuous Galerkin techniques. Standard DG methods have each degree of freedom is coupled with all other degree of freedoms on a neighboring element. By introducing additional unknowns along element interfaces, the HDG method is able to eliminate all degrees of freedom that do not reside on the interfaces. As such, a significantly smaller linear system is generated, and HDG gains much of its efficiency at higher orders ([29]). It turns out that the HDG method also has a number of attractive properties, namely, the capability of efficient implementations, optimal convergence rates in the potential and flux variables, as well as the availability of a post processing technique that results in the superconvergence of said variables. HDG methods are a subset of DG methods, so they sill retain local mass balance, and the discontinuous nature o f the solution variables. A thorough analysis of HDG methods can be found in [4], [5], and [14].
A number of works are available on multigrid for DG methods. Interior penalty methods are the most commonly analyzed, for instance, see [12], [11], [16], [2], and [1]. Most of these works are theoretical, and while they are able to prove convergence, the numerical experiments show rates below what is typically expected from GMG in this model setting. For the interior penalty class of DG methods, it was found that specialized smoothers and careful tuning of the stability parameter was required for better convergence results (see [26], [25]). It should be noted that these works explored stability parameters that resulted in a stable discretization, but GMG methods were not effective. Local Fourier analysis (local mode analysis, [10]) was applied to interior penalty DG methods in [20], [22], and [21]. However, even with specialized smoothers and careful parameter tuning, convergence rates were in the range of 0.4 to 0.6 for low order discretiztions ($p \leq 2$). In addition, local Fourier analysis is applied to a two level multigrid scheme, and is used as a heuristic to estimate GMG performance. Further, it is well known that two grid optimality does not always imply V cycle optimality (see [33]).

It should be noted that the HDG class of discretizations is quite large, similiar to that of standard DG methods. In [5], a unified framework is developed (similar to the work of Arnold et al. in [3]) to create a taxonomy of HDG methods. For instance, one can obtain HDG methods by utilizing one of: Raviart–Thomas DG, Brezzi–Douglas–Marini DG, Local DG, or Interior Penalty DG. In this work we have no need to distinguish between the various HDG methods, because we employ the LDG family of hybridizable methods ([14]). As such, since the LDG method is a dual or mixed technique, one needs to reformulate the underlying equation (1) as a first order system by introducing an auxiliary variable $\mathbf{q}$:

$$\mathbf{q} = -\mathbf{K}\nabla u, \quad \text{in} \quad \Omega, \tag{3}$$

$$\mathbf{K}^{-1}\mathbf{q} + \nabla u = 0, \quad \text{in} \quad \Omega,$$

$$\nabla \cdot \mathbf{q} = f, \quad \text{in} \quad \Omega,$$

$$u = g_D, \quad \text{on} \quad \partial\Omega. \tag{4}$$

We now describe the HDG method. Let $\mathcal{E}_h$ be a subdivision of Ω, made of quadrilaterals, K, of maximum diameter h. The unit normal vector outward of K is denoted

by $\boldsymbol{n}_K$. The mesh skeleton is denoted by Γ_h, that is the union of all the edges. We further decompose

$$\Gamma_h = \Gamma_h^o \cup \Gamma_h^\partial,$$

where Γ_h^∂ denote the set of all edges on the boundary of the domain, and Γ_h^o the set of all interior edges. The broken Sobolev space is denoted by $H^1(\mathcal{E}_h)$; it consists of piecewise H^1 functions on each mesh element. We use the following short-hand notation for L^2 inner-product on mesh elements and edges:

$$(u,v)_{\mathcal{E}_h} = \sum_{K\in\mathcal{E}_h} \int_K uv\,dx, \quad \langle u,v\rangle_{\Gamma_h} = \sum_{K\in\mathcal{E}_h} \int_{\partial K} uv\,ds, \qquad \forall u,v\in H^1(\mathcal{E}_h), \tag{5}$$

$$\langle \boldsymbol{w}\cdot\boldsymbol{n}, v\rangle_{\Gamma_h} = \sum_{K\in\mathcal{E}_h} \int_{\partial K} \boldsymbol{w}|_K\cdot\boldsymbol{n}_K v|_K ds, \qquad \forall(\boldsymbol{w},v)\in H^1(\mathcal{E}_h)^2\times H^1(\mathcal{E}_h). \tag{6}$$

The underlying approximation spaces for the HDG method are as follows:

$$\begin{aligned} W_h &= \{w\in L^2(\Omega):\, w|_K\in\mathbb{Q}_p(K)\ \ \forall K\in\mathcal{E}_h\}, \quad \boldsymbol{V}_h = W_h\times W_h,\\ M_h &= \{\mu\in L^2(\Gamma_h):\, \mu|_e\in\mathbb{Q}_p(e)\ \ \forall e\in\Gamma_h\},\\ M_h^0 &= \{\mu\in M_h:\, \mu=0 \text{ on } \partial\Omega\}, \end{aligned}$$

where $\mathbb{Q}_p(K)$ is the standard finite element space for quadrilaterals. That is, $\mathbb{Q}_p(K)$ is the tensor product of polynomials of degree p on each variable. The same definition (6) applies to $\langle \boldsymbol{w}\cdot\boldsymbol{n},\mu\rangle_{\Gamma_h}$ for functions $\boldsymbol{w}\in H^1(\mathcal{E}_h)^2$ and $\mu\in M_h$. We can now state the HDG method. We seek an approximation $(\boldsymbol{q}_h,u_h,\lambda_h)\in\boldsymbol{V}_h\times W_h\times M_h$ of the exact solution $(\boldsymbol{q}|_\Omega, u|_\Omega, u|_{\Gamma_h\setminus\partial\Omega_D})$ such that

$$(\boldsymbol{q}_h,\boldsymbol{v})_{\mathcal{E}_h} - (u_h,\nabla\cdot\boldsymbol{v})_{\mathcal{E}_h} + \langle\lambda_h,\boldsymbol{v}\cdot\boldsymbol{n}\rangle_{\Gamma_h^o} = -\langle P_h g_D,\boldsymbol{v}\cdot\boldsymbol{n}\rangle_{\partial\Omega}, \tag{7}$$

$$-(\boldsymbol{q}_h,\nabla w)_{\mathcal{E}_h} + \langle\boldsymbol{q}_h\cdot\boldsymbol{n},w\rangle_{\Gamma_h} + \langle\tau(u_h-\lambda_h),w\rangle_{\Gamma_h^o} + \langle\tau u_h,w\rangle_{\partial\Omega} = (f,w)_{\mathcal{E}_h} + \langle\tau P_h g_D,w\rangle_{\partial\Omega}, \tag{8}$$

$$\langle\boldsymbol{q}_h\cdot\boldsymbol{n},\mu\rangle_{\Gamma_h} + \langle\tau(u_h-\lambda_h),\mu\rangle_{\Gamma_h^o} + \langle\tau u_h,\mu\rangle_{\partial\Omega} = \langle\tau P_h g_D,\mu\rangle_{\partial\Omega}, \tag{9}$$

for all $(\boldsymbol{v},w,\mu)\in\boldsymbol{V}_h\times W_h\times M_h$. The factor τ is a local stabilization term that is piecewise constant on $\mathcal{E}_h$. In the above, $P_h g_D$ is the L^2–projection of g_D, defined by:

$$\int_e P_h g_D\,\mu = \int_e g_D\,\mu, \quad \forall\mu\in\mathbb{Q}_p(e), \quad \forall e\in\partial\Omega.$$

Introducing additional unknowns at first glance does not appear to add much benefit. However, the HDG method allows us to eliminate the unknowns $\boldsymbol{q}_h$ and u_h using equations (7) and (8) in an element by element manner to arrive at a weak formulation in terms of λ_h only. Let $\boldsymbol{Q}$, U and $\boldsymbol{\Lambda}$ denote the vectors of degrees of freedom for the numerical solutions $\boldsymbol{q}_h$, u_h and λ_h respectively. Since the solutions $\boldsymbol{q}_h$ and u_h are discontinuous, we can denote by $\boldsymbol{Q}_K$ and $\boldsymbol{U}_K$ the part of the vectors $\boldsymbol{Q}$, $\boldsymbol{U}$ corresponding to the degrees of freedom located on K. We also denote by $\boldsymbol{\Lambda}_K$ the part of the vector $\boldsymbol{\Lambda}$ that corresponds to the degrees of freedom located on the boundary of K.

We can express equations (7), (8) in matrix form:

$$\boldsymbol{A}_K\boldsymbol{Q}_K - \boldsymbol{B}_K^T\boldsymbol{U}_K + \boldsymbol{C}_K\boldsymbol{\Lambda}_K = \boldsymbol{R}_K, \quad \forall K\in\mathcal{E}_h, \tag{10}$$

$$\boldsymbol{B}_K\boldsymbol{Q}_K + \boldsymbol{D}_K\boldsymbol{U}_K + \boldsymbol{E}_K\boldsymbol{\Lambda}_K = \boldsymbol{F}_K, \quad \forall K\in\mathcal{E}_h. \tag{11}$$

We can solve for the local degrees of freedom:

$$\begin{bmatrix} \boldsymbol{Q}_K \\ \boldsymbol{U}_K \end{bmatrix} = \begin{bmatrix} \boldsymbol{A}_K & -\boldsymbol{B}_K^T \\ \boldsymbol{B}_K & \boldsymbol{D}_K \end{bmatrix}^{-1} \left(- \begin{bmatrix} \boldsymbol{C}_K \\ \boldsymbol{E}_K \end{bmatrix} \boldsymbol{\Lambda}_K + \begin{bmatrix} \boldsymbol{R}_K \\ \boldsymbol{F}_K \end{bmatrix} \right). \tag{12}$$

The above inverse is well defined and elimination of the local degrees of freedom for $\boldsymbol{q}$ and u can be done in parallel, independently of one another [5].

Equation (9) can be written in matrix form. Fix an interior edge $e \in \Gamma_h$ that is shared by elements K_1 and K_2 and denote by $\boldsymbol{\Lambda}_{K_1K_2}$ the set of degrees of freedom for the unknown λ_h, that lies on the edge e.

$$\boldsymbol{G}_{K_1e}\boldsymbol{Q}_{K_1} + \boldsymbol{G}_{K_2e}\boldsymbol{Q}_{K_2} + \tau\boldsymbol{H}_{K_1e}\boldsymbol{U}_{K_1} + \tau\boldsymbol{H}_{K_2e}\boldsymbol{U}_{K_2} - 2\tau\boldsymbol{M}_e\boldsymbol{\Lambda}_{K_1K_2} = \boldsymbol{0} \tag{13}$$

If the edge e lies on the boundary $\partial\Omega \cap \partial K$ we have

$$\boldsymbol{G}_{Ke}\boldsymbol{Q}_K + \tau\boldsymbol{H}_{Ke}\boldsymbol{U}_K = \boldsymbol{S}_e$$

Now we summarize a general HDG assembly and solve procedure:

(i) Use equation (12) to assemble the local problems for $\boldsymbol{\Lambda}_K$.

(ii) Assemble the local problems for $\boldsymbol{\Lambda}_K$ into a global system matrix using (13).

(iii) Solve the global system matrix for Λ.

(iv) Using the newly solved for Λ in equation (13), reconstruct $\boldsymbol{U}$ and $\boldsymbol{Q}$.

(v) Postprocess U and Q to obtain superconvergence.

Step (ii) in our framework is technically assembly free, we do not directly assemble and store the global system matrix. In stead, we exploit the unassembled local problems from equation (12) to generate a matrix vector multiplication routine. The resulting routine utilizes many small but dense matrices, instead of a large sparse matrix. See [37] for a survey of different finite element assembly strategies (matrix free or otherwise). Although the focus of our paper is not assembly, an algorithm was developed in [28] to accelerate the assembly of the HDG global system matrix.

Step (ii) in our framework is technically assembly free, we do not directly assemble and store the global system matrix. In stead, we exploit the unassembled local problems from equation (12) to generate a matrix vector multiplication routine. The resulting routine utilizes many small but dense matrices, instead of a large sparse matrix. See [37] for a survey of different finite element assembly strategies (matrix free or otherwise). Although the focus of our paper is not assembly, an algorithm was developed in [28] to accelerate the assembly of the HDG global system matrix.

4. Basis functions. We select a tensor product basis for the space Q^p. To easily facilitate high order approximations, we invoke a nodal basis with nodes that correspond to roots of Jacobi polynomials (Gauss-Legendre-Lobotto (GLL)).

Various operators in the HDG discretization and multigrid method may require evaluation of the nodal basis at points that are not be nodal. Some authors use a modal–nodal transformation to deal with such evaluations (see [27]). Another technique is to simply use the fast and stable *barycentric interpolation* (see [8]). This allows one to stay in the Lagrange basis and not have to resort to a generalized Vandermonde matrix. That is, barycentric interpolation allows for the stable evaluation of the Lagrange basis at any point in its domain. Given N grid points, the setup cost is $\mathcal{O}(N^2)$ to generate the barycentric weights, and a $\mathcal{O}(N)$ cost for each evaluation. Due do the discontinuous nature of the HDG approximation, evaluations occur elementwise, so the stability of barycentric interpolation is perhaps more important than cost of revaluation. Since barycentric interpolation allows for arbitrary evaluation, one can select quadrature points that differ from the nodal interpolation points.

Let v be a polynomial v of degree n, that interpolates scattered data $\{f_0, f_1, \ldots, f_n\}$

through the points $\{x_0, x_1, \ldots, x_n\}$. The second form of the barycentric formula is

$$v(x) = \frac{\sum_{j=0}^{n} \frac{W_j}{x - x_j} f_j}{\sum_{j=0}^{n} \frac{W_j}{x - x_j}},$$

where W_j are the barycentric weights. Interpolation points x_i arising from classical orthogonal polynomials have explicit formulas for their barycentric weights ([35], [38]). For instance, for GLL and GL nodes,

$$W_j^{\mathrm{GLL}} = (-1)^j \sqrt{w_j^{\mathrm{GL}}} \quad W_j^{\mathrm{GL}} = (-1)^j \sqrt{x_j^{\mathrm{GL}} w_j^{\mathrm{GL}}},$$

where w_j and x_j (with appropraite superscripts) are the GL and GLL quadrature nodes and weights.

The reference element is discretized using a tensor product of a 1D spectral grid. In the case of GLL nodes, $\{\xi_i^{(p)}\}_{i=0}^{p}$ are zeros of a particular family of Jacobi polynomials (see [27]), where $-1 \leq \xi_i \leq 1$. We adopt the standard that the reference element in is $[-1, 1]$ in 1D, $[-1, 1] \times [-1, 1]$ in 2D, and $[-1, 1] \times [-1, 1] \times [-1, 1]$ in 3D. See Fig. 1 for a sample spectral grid. In 2D the tensor product can be written as

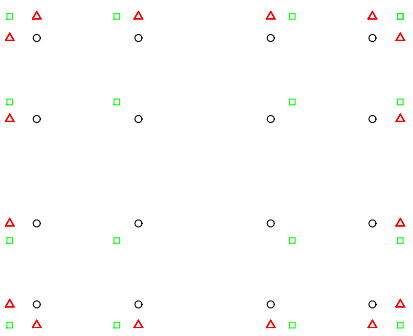


FIG. 1. *GLL nodes (open squares), GL nodes (open circles), GL surface nodes (open triangles).*

$$(\xi_i, \eta_j) \stackrel{def}{=} (\xi_i^{(p)}, \eta_j^{(p)}), \quad i, j = 0, 1, \ldots, p,$$

where p is the polynomial degree associated with Q^p. On each element, we define the basis functions as tensor products:

$$\phi_I^e(\xi, \eta) = \ell_i(\xi)\ell_j(\eta),$$

with $I = i + j(P+1)$ (lexicographic ordering, and ℓ_i is the Lagrange polynomial that is nodal at GLL node i). In one dimension, if there are $p+1$ GLL nodes, then there are $p+1$ associated basis functions. In two dimensions, we take the tensor product of the 1D GLL nodes, which results in $(p+1)^2$ GLL nodes, and $(p+1)^2$ associated basis functions. In the index I, if $i = j = p$, then $I = p + p(p+1) = p^2 + 2p$, and including the index contribution from $i = j = 0$, we have a total of $p^2 + 2p + 1 = (p+1)^2$ basis functions in 2D as well.

For brevity, let the lexicographic ordering $I = i + j(p+1) =: (i,j)$. Let the reference element be given by $\square = [-1,1] \times [-1,1]$. Just as in the 1D case, we assume that

$$\begin{aligned} u(x,y)\Big|_{\Omega^e} &\approx \sum_{i=0}^{p}\sum_{j=0}^{p} u_{ij}^e \phi_I^e(\xi,\eta), \quad (\xi,\eta) \in \square, \\ \phi_I^e(\xi_m,\eta_n) &= \delta_{im}\delta_{jn}, \quad i,j,n,m \in \{0,\dots p\}, \\ (\xi,\eta) &= \text{ GLL quadrature points } \in \square. \end{aligned}$$

Using reference elements is standard in finite element methods, and we use them here. We have

$$\begin{aligned} \iint_{\square} u(\xi,\eta)\, d\xi d\eta &= \int_{-1}^{1}\left(\int_{-1}^{1} u(\xi,\eta)\, d\xi\right) d\eta \\ &\approx \sum_{i=0}^{P}\sum_{j=0}^{P} w_i w_j u(\xi_i,\eta_j), \end{aligned}$$

where w_i, w_j are the GLL quadrature weights in the x and y directions. We want to map an arbitrary element to this element, which requires a change of variables $\xi = \xi(x,y)$, $\eta = \eta(x,y)$. The Jacobian of this mapping is

$$\mathcal{J}^e(\xi,\eta) = \begin{vmatrix} \frac{\partial x}{\partial \xi} & \frac{\partial x}{\partial \eta} \\ \frac{\partial y}{\partial \xi} & \frac{\partial y}{\partial \eta} \end{vmatrix} = \left| \frac{\partial x}{\partial \xi}\frac{\partial y}{\partial \eta} - \frac{\partial y}{\partial \xi}\frac{\partial x}{\partial \eta} \right|.$$

Hence, integrating over an arbitrary element Ω^e $(\cup_{e=1}^{E}\Omega^e)$,

$$\begin{aligned} \iint_{\Omega^e} u(x,y)\, dx\, dy &= \iint_{\square} u^e(\xi,\eta)\mathcal{J}^e(\xi,\eta)\, d\xi\, d\eta \\ &\approx \sum_{i=0}^{p}\sum_{j=0}^{p} w_i w_j u^e(\xi_i,\eta_j)\mathcal{J}_{ij}^e, \end{aligned}$$

where $\mathcal{J}^e(\xi_i,\eta_j) = \mathcal{J}^e(\xi,\eta)$.

The above approach assumed that the interpolation points and quadrature points were the same (classic spectral element method). However, GLL quadrature rule for $p+1$ points is only exact for polynomials of degree $2p-1$. If higher order quadrature is needed, the GL quadrature rule for $p+1$ points is exact for polynomials of degree $2p+1$. Then, one can fix the GLL interpolation points, and evaluate the Lagrange basis functions at GL quadrature points if desired. To do this, we utilize the barycentric formula:

$$\ell_j(x) = \frac{\dfrac{W_j^{\mathrm{GL}}}{x - x_j^{\mathrm{GLL}}}}{\displaystyle\sum_{k=0}^{p} \frac{W_k^{\mathrm{GL}}}{x - x_k^{\mathrm{GLL}}}}.$$

Barycentric interpolation enables the stable and fast evaluation of the Lagrange polynomial basis ℓ_j anywhere in its domain.

5. HDG discretization. Before proceeding with the results of the HDG GMG method, we verify numerically that the HDG discretization provides the expected optimal L^2 convergence rates; $p+1$ for *both* the potential u_h and its flux $\boldsymbol{q}_h$. Moreover, with the use of a local postprocessing filter ([4]), we can achieve superconvergence of

the potential u_h, so that it converges in the L^2 norm with the rate $p+2$. HDG does fall under the umbrella of stabilized DG methods, so the parameter τ in equations (8) (9) and needs to be specified. The local stability parameter is piecewise constant, defined facet-by-facet. For the model problem we set $\Omega = [0,1] \times [0,1]$, $\partial\Omega_D = \partial\Omega$ ($\partial\Omega_N = \emptyset$), $\mathbf{K}(x,y) = \tanh(x+y) + 1$. The domain Ω is partitioned into $N \times N$ squares. A manufactured solution is used to examine the error: $u(x,y) = x(x-1)y(y-1)\exp(-x^2-y^2)$, and the corresponding forcing function f is determined from u.
For the model Poisson problem it turns out that $\tau \equiv 1$ on every facet provides optimal convergence rates. In Table (1), it is apparent that the expected convergence rates are met. Moreover, the postprocessed potential u_h^* results in a rate of $p+2$. The postprocessed flux $\boldsymbol{q}_h^*$ converges in a rate of $p+1$, but the errors are smaller than that of $\boldsymbol{q}_h$.

6. Multigrid convergence. Fig. 2 and Fig. 3 display the results for $2 \leq p \leq 8$ and fine mesh with $(2^4)^2$ elements. We employ a FSAI smoother on each level, with $\nu_1 = \nu_2 = 2$ pre and post smoothing steps. The FSAI smoother is constructed so that it results in an approximate inverse with a operator complexity of unity (nnz[$\boldsymbol{M}$]=nnz[$\boldsymbol{A}$]). Subspace non-inheritance is used to generate coarse grid operators. For the p-GMG phase of the GMG method, we use full arithmetic coarsening. To measure the convergence rate, we keep track of the two norm of the fine grid residual between successive iterations:

$$\rho_k = \frac{\|(r^h)^{(k)}\|_2}{\|(r^h)^{(k-1)}\|_2}.$$

We can see that the results are quite good, even for a modest FSAI smoother. All of the convergence rates are under 0.22, and tend to cluster in the range 0.13-0.15 for even polynomial degrees. This observation perhaps indicates that even-p coarsening or geometric-p will not only be more efficient (more rapid coarsening), but also more accurate.
With a more aggressive smoothing, one might expect even better convergence results from the HDG GMG method. For the next set of experiments, we allow the FSAI operator complexity to grow ((nnz[$\boldsymbol{M}$] $\approx$ 2.5 nnz[$\boldsymbol{A}$]). In figures 7, 8, 9, and 10, display the results of this change. For the V-cycle, the residual is reduced to machine precision after only 5 to 7 iterations (Fig. 7). The aggressive smoothing also yields stellar convergence rates, below 0.015, as can be ascertained from Fig. 8. The performance of the V-cycle is better than ideal, so we can easily extend our HDG GMG method to leverage the *full multigrid cycle* (FMG). FMG is well known to be *the* optimal multigrid schedule for linear problems; with a single FMG iteration, the residual is reduced to discretization level error. Moreover, it only requires $\mathcal{O}(N)$ floating point operations to achieve this accuracy ([9], [10], [36]). Indeed, Fig. 9 numerically verifies that a *single* FMG iteration is enough to reach discretization level error. There is something that particularly interesting about Fig. 9 - the FMG iteration performs better for higher orders $6 \leq p$. This second experiment of course comes at a price: the FSAI smoothers on each level have an operator complexity in the rage of 2.65 to 2.85. Such a trade off may be valuable for problems with highly varying or discontinuous coefficients. Also, the FSAI smoother can be easily tuned to control how aggressively its operator complexity grows.

7. Performance model. The storage and assembly of global matrices in finite element methods can be exceedingly prohibitive, especially at higher orders. By leveraging matrix free algorithms, one can save on memory, and, convert a memory bound problem (sparse matrix vector multiplication) into a compute bound problem. The authors in [34] found that to improve sparse matrix vector multiplication for HDG methods, specialized storage formats were needed. Since the HDG method can be reduced to a problem on the trace space, this allows for a assembly (matrix free or otherwise) of the discretization operator in a facet-by-facet manner, instead of a element-by-element manner. The importance of this is that the element-by-element approach requires a synchronization (barrier, atomic, etc.) in order to avoid race conditions. A graph coloring algorithm is typically used in this situation, but

		$\|u_h - u\|_{L^2(\Omega)}$		$\|u_h^* - u\|_{L^2(\Omega)}$		$\|q_h - q\|_{L^2(\Omega)}$		$\|q_h^* - q\|_{L^2(\Omega)}$	
p	N	Error	Rate	Error	Rate	Error	Rate	Error	Rate
6	2	1.05e-07	-	3.02e-09	-	2.39e-07	-	1.12e-07	-
	4	8.51e-10	6.95	1.13e-11	8.06	1.95e-09	6.94	8.41e-10	7.06
	8	6.97e-12	6.93	4.45e-14	7.99	1.61e-11	6.92	6.65e-12	6.98
	16	5.60e-14	6.96	7.12e-16	5.97	1.41e-13	6.84	6.36e-14	6.71
	32	8.79e-16	5.99	5.35e-16	4.12	2.11e-13	-5.82	1.47e-13	-1.21
5	2	7.53e-07	-	3.50e-08	-	1.77e-06	-	9.80e-07	-
	4	1.69e-08	5.47	3.03e-10	6.85	3.91e-08	5.50	1.81e-08	5.76
	8	2.90e-10	5.87	2.43e-12	6.96	6.72e-10	5.86	2.95e-10	5.94
	16	4.72e-12	5.94	1.92e-14	6.98	1.10e-11	5.94	4.69e-12	5.97
	32	7.52e-14	5.97	3.68e-16	5.70	2.05e-13	5.74	1.01e-13	5.53
4	2	1.46e-05	-	7.04e-07	-	3.32e-05	-	1.79e-05	-
	4	4.92e-07	4.89	1.09e-08	6.02	1.13e-06	4.88	5.49e-07	5.03
	8	1.65e-08	4.90	1.73e-10	5.97	3.80e-08	4.89	1.76e-08	4.96
	16	5.37e-10	4.94	2.75e-12	5.98	1.24e-09	4.94	5.62e-10	4.97
	32	1.71e-11	4.97	4.33e-14	5.99	3.98e-11	4.97	1.78e-11	4.98
3	2	8.61e-05	-	7.58e-06	-	2.05e-04	-	1.35e-04	-
	4	7.78e-06	3.47	2.63e-07	4.85	1.80e-05	3.52	9.86e-06	3.78
	8	5.49e-07	3.82	8.63e-09	4.93	1.27e-06	3.82	6.59e-07	3.90
	16	3.63e-08	3.92	2.77e-10	4.96	8.43e-08	3.91	4.28e-08	3.95
	32	2.33e-09	3.96	8.80e-12	4.98	5.43e-09	3.96	2.73e-09	3.97

TABLE 1
Errors and convergence rates of the HDG scheme, on a Cartesian mesh of $N \times N$ elements.

only allows for a group of colors to be utilized at any given time. The facet-by-facet approach does not depend on the vertex degree, but only on the element type (how many facets on a given element). Fig. 12 displays some example DOFs and connectivity for HDG, DG, and continuous Galerkin methods.

The HDG method is able to exploit facet-by-facet connectivity; given an interior facet, only the two elements that share said facet will contribute to its DOFs. An element-to-facet mapping will allow one to gather and scatter the DOFs on a facet. Pseudocode for our matrix vector multiply routine is given in Algorithm (1).

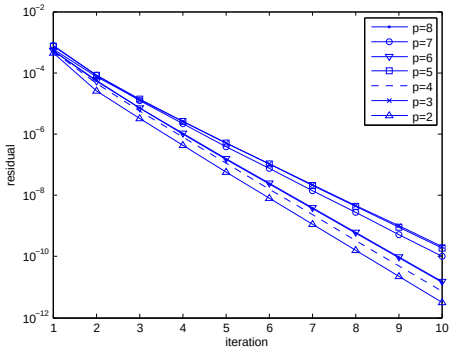


FIG. 2. *V-Cycle.*

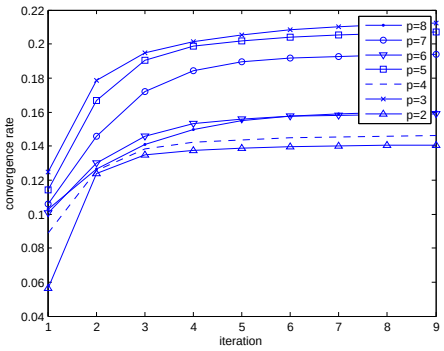


FIG. 3. *Convergence rate (V-Cycle).*

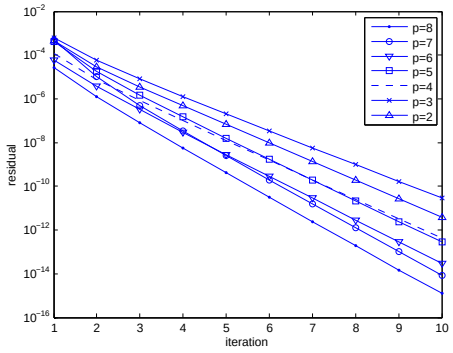


FIG. 4. *W-Cycle.*

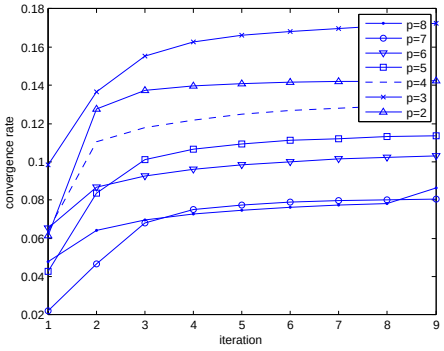


FIG. 5. *Convergence rate (W-Cycle).*

FIG. 6. *GMG for HDG (SPAI-1 smoother).*

7.0.1. Knights Landing (KNL) accelerator. In this work we use the second generation Xeon Phi designed by Intel®. It is a manycore processor, and the 7000 series has anywhere from 64 to 72 cores, with 4 threads per core. The KNL is an example of a high throughput low memory device. The design of the KNL is similar to other accelerators: a large number of cores with lower clock speeds are packed into the unit, enabling a large vector width, as well as having access to a user-manageable fast memory hierarchy. One interesting feature of the KNL is that it can be programmed using traditional parallel paradigms like OpenMP, MPI, and pthreads. Further details regarding the KNL can be obtained in [24]. For specifications of the KNL used in this work, see Table 2.

7.0.2. Local matrix generation. In order to make use of the assembly free matrix vector multiply, one needs to generate the associated local matrices (in the case of HDG, see (i), (iv), and (v) in Section 3). To generate the required local matrices, for simplicity, we use a one core (thread) per element strategy. However, as one increases the polynomial order (beyond $p = 5$), this strategy loses performance. Further improvements may potentially be obtained by utilizing nested parallelism, or linear algebra libraries dedicated for small or medium sized matrices ([19], [28]).

For all computational experiments we set the KNL in quadrant mode. Figures 13 and 13 display the wall clock and speed up as the polynomial order and number of

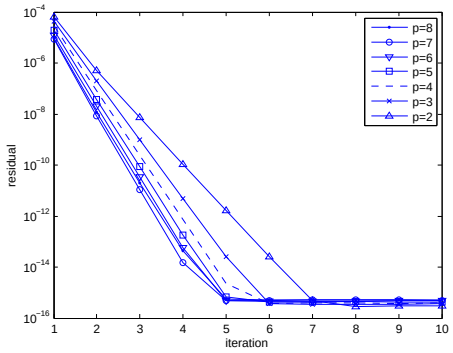


FIG. 7. *V-Cycle.*

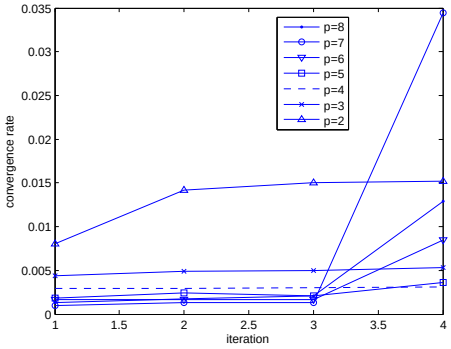


FIG. 8. *Convergence rate.*

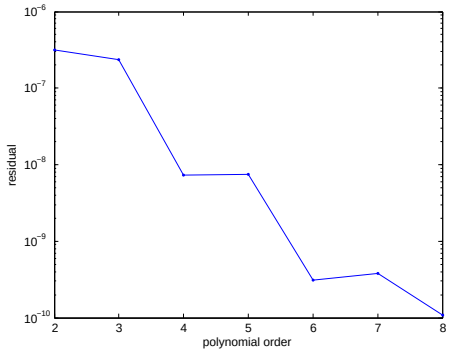


FIG. 9. *FMG-Cycle.*

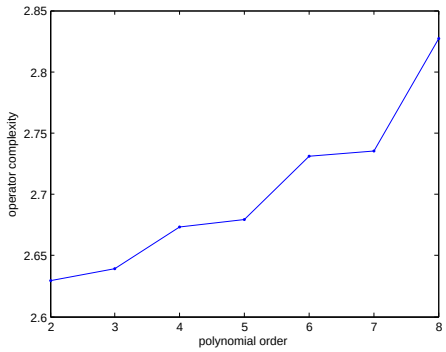


FIG. 10. *Operator complexity.*

FIG. 11. *FMG for HDG (aggressive FSAI).*

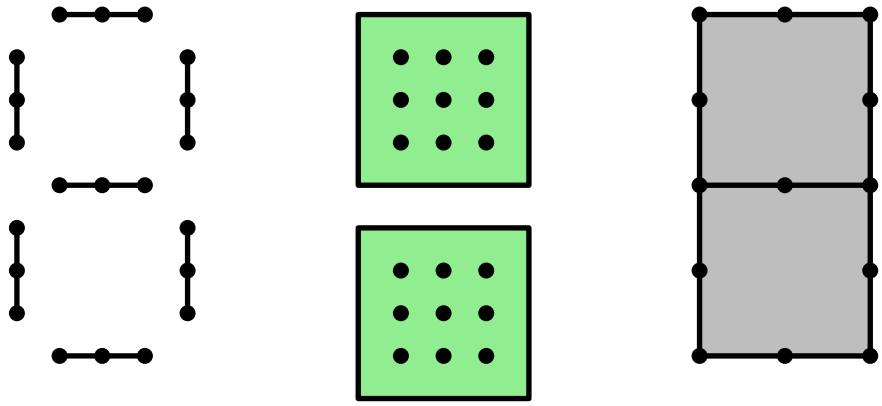


FIG. 12. *HDG, DG, and continuous Galerkin DOFs.*

threads is varied. In Fig. 13, one can see that as we increase the number of threads

Algorithm 1 Algorithm for HDG matrix vector multiply (assembly free).

procedure MATRIX VECTOR MULTIPLY
    Algorithm for HDG matrix vector multiply (assembly free)
    Load $\mathbb{K}_K$ and x
    eN $\leftarrow$ Map0(tid) (Given thread id, find the edge it corresponds to)
    (E1,E2) $\leftarrow$ Map1(eN) (Find the elements that share edge).
    (idx1) $\leftarrow$ Map2(eN,E1) (load local index of DOF on element 1)
    (idx2) $\leftarrow$ Map3(eN,E2) (load local index of DOF on element 2)
    $y_1 = 0$
    for $j = 1$ to size($\mathbb{K}_{\text{E1,eN}}$) **do**memo
        $y_1 \leftarrow y_1 + \mathbb{K}_{\text{E1,eN}}(\text{ idx1(j)}, :) \cdot x$
    $y_2 = 0$
    for $j = 1$ to size($\mathbb{K}_{\text{E1,eN}}$) **do**
        $y_2 \leftarrow y_2 + \mathbb{K}_{\text{E1,eN}}(\text{ idx1(j)}, :) \cdot x$
    $y(\text{tid}) = y_1 + y_2$

beyond 32 or 64, diminishing returns are very noticeable. To generate the results of Fig. 13, we fix the number of threads to 64, and we vary the polynomial order. The comparison with a serial implementation clearly shows that the additional parallelism the KNL offers is beneficial; speed ups ranging from 2X to 32X are attained.

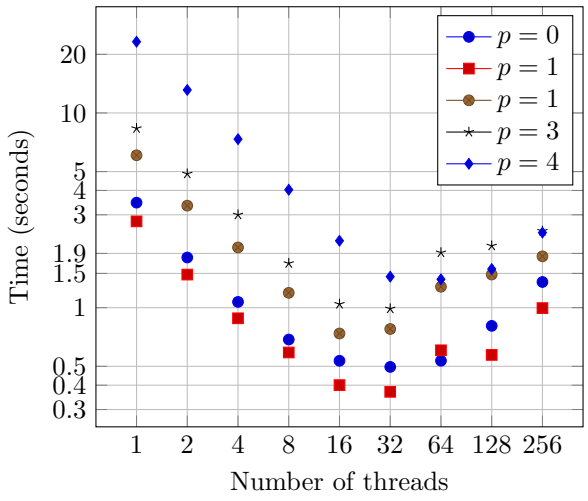


FIG. 13. *Local matrix assembly wall clock.*

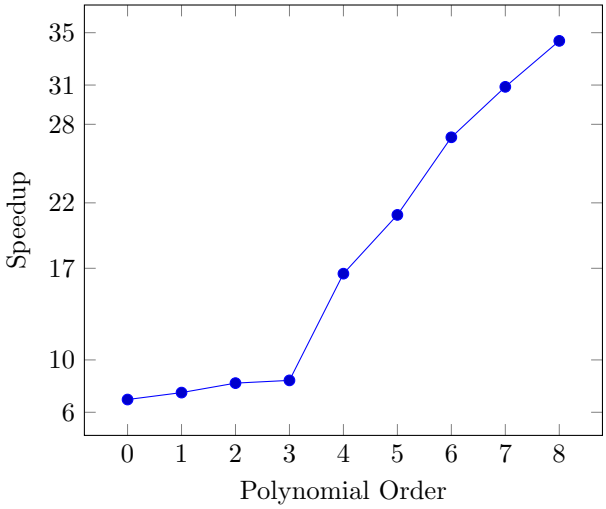


FIG. 14. *Local matrix assembly speedup.*

7.1. Roofline analysis. The KNL (7210 processor number) used in this analysis runs at 2.1 GHz (double precision), a double precision processing power of of 2,199 GFLOP/s, and the STREAM memory bandwidth benchmark (triad, see [32]) reports a bandwidth of 300 GB/s. These two metrics provide the performance boundaries for arithmetic throughput and memory bandwidth limits, respectively. For our numerical experiments, the clustering mode is set as quadrant, and cache mode is set to flat. Further details about our testbed can be obtained in table 2.

7.1.1. Sparse Matrix Vector Multiply (SPMV). In general matrix vector multiplication is severely bandwidth bound. For a dense matrix of dimension N, we

Processor number	7210
# of cores	64
Processor base frequency	1.30 GHz
Cache	32 MB L2
RAM	384 GB DDR4
MCDRAM	16 GB
Instruction set	64-bit
Instruction set extension	Intel® AVX-512
Operating system	CentOS 7.2
Compiler	Intel® Parallel Studio XE 16

TABLE 2
KNL testbed specifications.

can expect $(2N^2 - N)$ FLOPs and $8(2N^2 + N)$ MEMOPs (bytes). Thus, for large N, we can expect an arithmetic intensity of only $\lim_{N\to\infty}(2N^2-N)/(8(2N^2+N)) = 0.25$. For finite element problems the underlying discretization matrix is typically sparse, which pushes SPMV further into the bandwidth bound region on the roof line chart. Matrix free operation can remedy this situation by improving the arithmetic intensity. That is, matrix free application can shift our bandwidth bound SPMV to a compute bound (or less bandwidth bound) problem.

In Fig. 16, we include a roofline analysis of our matrix vector multiply routine; in the context of its performance on the Laplacian operator. The roofline analysis ([40]) allows us to identify bottlenecks, confirm algorithm limitations, as well as gives us insight on what we should focus on in terms of optimization. Two different techniques are studied. Sparse matrix storage (CSR format, see Fig. 17) has the lowest performance arithmetic intensity, with a theoretical limit of 0.25. The multithreaded Intel MKL library is used for this approach ([23]). For high orders the HDG method has hundreds of nonzeros per row. Slightly better but similar performance was obtained in [34] by using a specialized block sparse matrix storage format on GPUs. The matrix free technique shifts the arithmetic intensity favorably (see 16); this behavior is typical in spectral methods and spectral elements, due to near constant FLOP and memory requirements per degree of freedom ([37], [13], [31]).

Fig. 15 displays the bandwidth measurements. For lower order polynomials ($p < 5$), we see that a maximum of 50% of the peak bandwidth is achieved. The overhead costs of forking threads is dominating computations for $p < 5$. As p increases beyond five, the cost of forking threads is no longer dominant. Our algorithm reaches 80% of the peak bandwidth reported by the STREAM benchmark.

7.1.2. Projection From Volume To Surface. The generation of the local solvers from equation (12) is completely data parallel and needs no synchronizations or thread communication. It requires a dense linear solve, two matrix vector multiplications and a single SAXPY.

Roughly the complexity model for this projection operator in terms of FLOPs can be estimated by

$$\begin{aligned} \text{FLOPs} &= 2N^2 - N + (2/3)N^3 + M(2N-1) + 2M), \\ \text{MEMOPs} &= 8(N^2 + NM + M + N). \end{aligned}$$

where $N = 4(p+1)^2$ and $M = 4*(k+1)$. Table (3) collects the results for polynomial order $0, 1, 2, 3$, and 4. As we increase the polynomial order, performance increases due

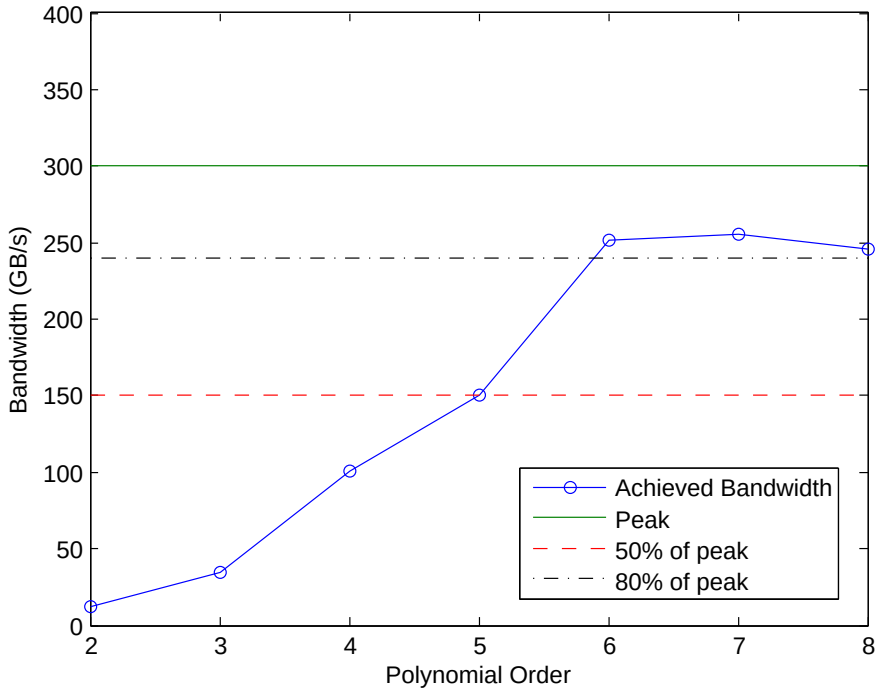


FIG. 15. *Achieved bandwidth vs polynomial order for matrix-free SPMV.*

to the very much compute bound nature of the local HDG solvers; as the dense linear solve dominates with $\mathcal{O}(n^3)$ FLOPs for $\mathcal{O}(n^2)$ data. We again note that the focus of this work is not on the generation of this local matrices, but we can still obtain reasonable performance utilizing a straightforward implementation.

After solving the trace space system given by equation (13), if the volume solutions u and $\boldsymbol{q}$ are desired, one can invoke equation (12) to reconstruct them. The cost of this proceedure depends on if one discards the local matrices. In this case, one has to recompute and the cost is the same as the projection from the surface to volume. If one instead keeps the local matrices, all that is needed is a two matrix vector multiplies and a single SAXPY. Ultimately this comes down to a preference of convenience over memory concerns.

Polynomial Order	AI	GFLOP/s
0	0.84	0.0032
1	1.41	1.15
2	2.60	5.129
3	4.33	21.18
4	6.56	53.66

TABLE 3
Arithmetic intensity and GFLOP/s for the local solvers.

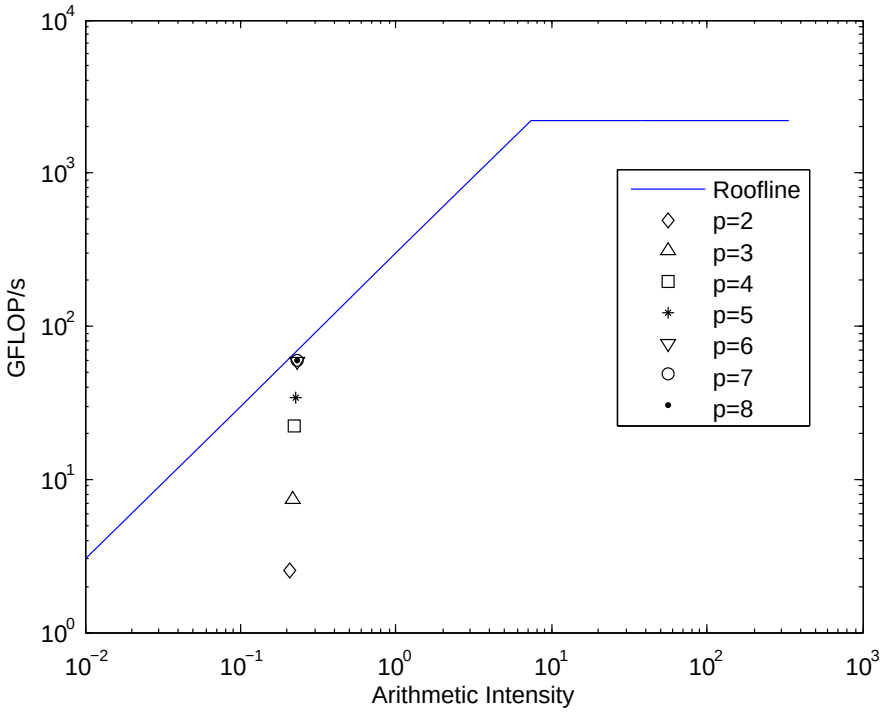


FIG. 16. *Roofline analysis for matrix-free SPMV.*

7.1.3. Local Postprocessing. In this section we study a work–precision diagram ([18]) to help us infer information about if or when we should consider postprocessing. For time dependent problems, postprocessing need not occur at each time step, but only when an enhanced solution is required. The postprocessing is completely data parallel and requires the assembly of a dense stiffness matrix (of size $(p+2)^2 \times (p+2)^2$ that is local to a given element), and dense linear solve. We define work as the number of FLOPs required by the postprocessing, and precision is the respective error of the postprocessing in the L^2-norm. Postprocessing in the case of $p = 0$ does not result in superconvergence, so we exclude that result. We consider the same model problem used in section (5).

For Fig. (18), each data point corresponds to a polynomial order on a fixed mesh. As we increase the polynomial order, we see spectral convergence as expected. The break-even mark emerges at around $p = 3$, and $p = 4$ is where postprocessing is preferable, which is inline with HDGs methods being more efficient for higher orders. One should note that in the case of time dependent problems, another consideration to take into account is there is an additional projection operation that is needed if postprocessing is used; because u_h has local dimension $(p+1)^2$ and u_h^* has local dimension $(p+2)^2$.

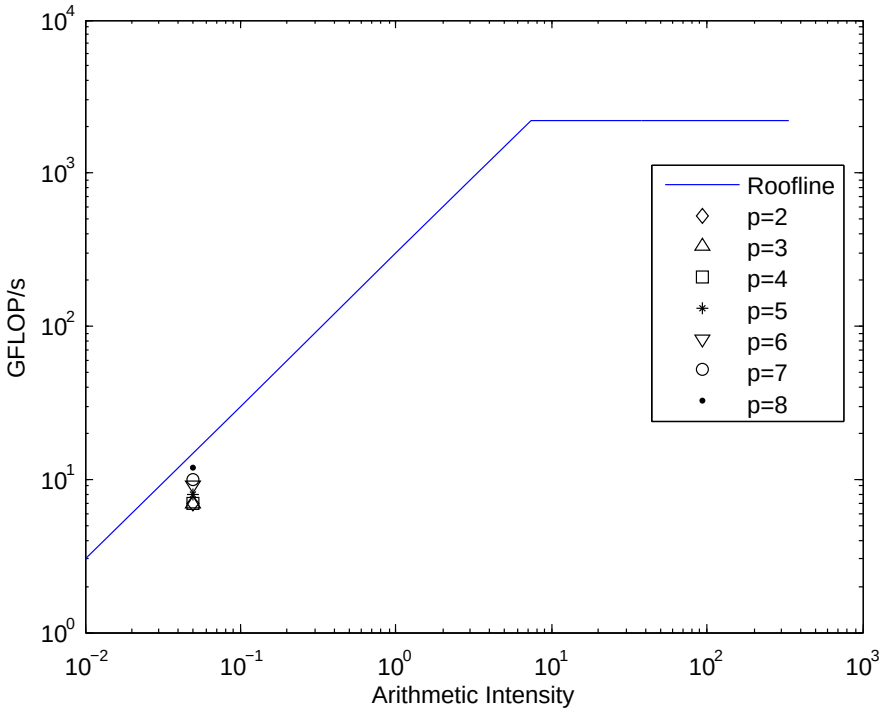


FIG. 17. *Roofline analysis for CSR SPMV.*

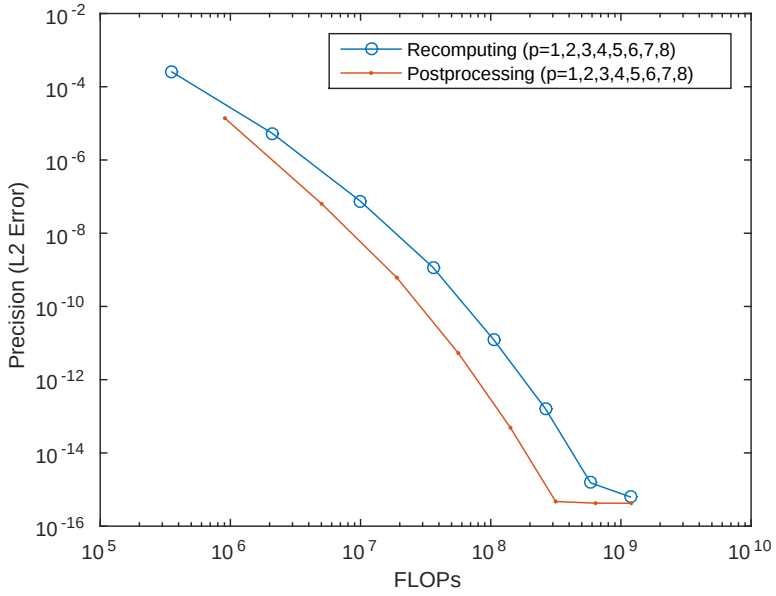


FIG. 18. *Work-precision diagram.*

8. Conclusions. We presented a highly efficient multigrid technique for solving a second order elliptic PDE with the HDG discretization. Both the multigrid technique and the HDG discretization are amenable to high throughput low memory environments like that provided by the KNL architecture; this was demonstrated by using thorough profiling and utilizing a roofline analysis ([40]). The HDG method has much of its computation localized due to the discontinuous nature of the solution technique. Moreover, the HDG method brings static condensation to DG methods, which significantly reduces the number of nonzeros in the discretization operator. This in turn means that less work is required for a linear solver to obtain solutions. Since the HDG method converges with optimal orders for all of the variables it approximates, a local element by element post-processing is available. A work precision diagram shows that for polynomial order $p \geq 3$, the post-processing is always worth it, as it is both cheaper and more accurate than reconstructing the problem one polynomial degree higher.

Any iterative solver will require sparse matrix vector multiplication (matrix free or otherwise); including multigrid. Two different approaches were examined to perform sparse matrix vector multiplication: matrix based and matrix free. For high order HDG, the matrix free technique better utilizes the resources available on the KNL, because of increased arithmetic intensity. In the high order regime, the matrix based technique requires a sparse matrix storage which increases time to solution. This is mainly because of additional assembly time, having a low arithmetic intensity, and erratic memory access patterns for sparse matrix vector multiplication.

Our algorithm is able to attain 80% of peak bandwidth performance for higher order polynomials. This is possible due to the data locality inherent in the HDG method. Very good performance is realized for high order schemes, due to good arithmetic intensity, which declines as the order is reduced. The performance is lower for polynomial order $p < 6$. With an approach similar to the work done in [30], where multiple cells are processed by a thread block, one would be able to increase performance for lower order polynomials. We observed speedups when compared to a multicore CPU for the HDG methods components, namely, volume to surface mapping, surface to volume mapping, matrix free matrix vector multiplication, local matrix generation, and the post-processing. The ratio of peak flop rates on the two target architectures was roughly 100X, and peak bandwidths was 5X, so this figures fits with our model of the computation.

This is possible due to the data locality inherent in the HDG method. Very good performance is realized for high order schemes, due to good arithmetic intensity, which declines as the order is reduced.

An attractive feature of the KNL is that it can be programmed with traditional parallel paradigms like OpenMP, MPI, and pthreads. This opens opportunities for acceleration of parallel software written in these traditional parallel paradigms with significantly limited intrusion.

Acknowledgments. Fabien acknowledges the support from the Ken Kennedy Institute and the Ken Kennedy–Cray graduate Fellowship. Fabien also acknowledges the support from the Richard Tapia Center for Excellence & Equity, and the Rice Graduate Education for Minorities program. This work used the Extreme Science and Engineering Discovery Environment (XSEDE), which is supported by National Science Foundation grant number ACI-1053575. The KNL used in this work was provided to the RELACS research group by Richard Mills (Intel® Parallel Computing Center at Rice University).

REFERENCES

[1] P. F. Antonietti, M. Sarti, and M. Verani, *Multigrid algorithms for hp-Discontinuous Galerkin discretizations of elliptic problems*, ArXiv e-prints, (2013).

[2] P. F. Antonietti, M. Sarti, and M. Verani, *Multigrid algorithms for high order discontinuous Galerkin methods*, Domain Decomposition Methods in Science and Engineering XXII, (2016), pp. 3–13.

[3] D. N. Arnold, B. C. F. Brezzi, and L. D. Marini, *Unified analysis of discontinuous Galerkin*

methods for elliptic problems, SIAM J. Numerical Analysis, 39 (2002), pp. 1749–1779.
[4] B. D. B. Cockburn and J. Guzmán, *A superconvergent ldg-hybridizable Galerkin method for second-order elliptic problems*, Math. Comput., 77 (2008), pp. 1887–1916.
[5] J. G. B. Cockburn and R. D. Lazarov, *Unified hybridization of discontinuous Galerkin, mixed, and continuous Galerkin methods for second order elliptic problems*, SIAM J. Numerical Analysis, 47 (2009), pp. 1319–1365.
[6] P. Bastian, *Load balancing for adaptive multigrid methods*, SIAM Journal on Scientific Computing, 19 (1998), pp. 1303–1321.
[7] P. Bastian, G. Wittum, and W. Hackbusch, *Additive and multiplicative multi-grid; a comparison*, Computing, 60 (1998), pp. 345–364.
[8] J. P. Berrut and L. N. Trefethen, *Barycentric lagrange interpolation*, SIAM Review, 46 (2004), pp. 501–517.
[9] A. Brandt, *Multi-level adaptive solutions to boundary-value problems*, Mathematics of Computation, 31 (1977), pp. 333–390.
[10] A. Brandt and O. Livne, *Multigrid Techniques: 1984 Guide with Applications to Fluid Dynamics, Revised Edition*, Classics in Applied Mathematics, Society for Industrial and Applied Mathematics, 2011.
[11] S. C. Brenner, J. Cui, and L.-Y. Sung, *Multigrid methods for the symmetric interior penalty method on graded meshes*, Numerical Linear Algebra with Applications, 16 (2009), pp. 481–501.
[12] S. C. Brenner and J. Zhao, *Convergence of multigrid algorithms for interior penalty methods*, Applied Numerical Analysis & Computational Mathematics, 2 (2005), pp. 3–18.
[13] C. Cantwell, S. Sherwin, R. Kirby, and P. Kelly, *From h to p efficiently: Strategy selection for operator evaluation on hexahedral and tetrahedral elements*, Computers & Fluids, 43 (2011), pp. 23–28.
[14] B. Cockburn, J. G. B. Dong, M. Restelli, and R. Sacco, *A hybridizable discontinuous Galerkin method for steady-state convection-diffusion-reaction problems*, SIAM J. Scientific Computing, 31 (2009), pp. 3827–3846.
[15] R. P. Fedorenko, *The speed of convergence of one iterative process*, U.S.S.R. Comput. Math. Math. Phys., 4 (1964), pp. 559–564.
[16] J. Gopalakrishnan and G. Kanschat, *A multilevel discontinuous Galerkin method*, Numerische Mathematik, 95 (2003), pp. 527–550.
[17] W. Hackbusch, *Multi-Grid Methods and Applications*, Springer Series in Computational Mathematics, Springer Berlin Heidelberg, 2013.
[18] E. Hairer, S. P. Norsett, and G. Wanner, *Solving Ordinary, Differential Equations I, Nonstiff problems*, 2Ed. Springer-Verlag, 2000.
[19] A. Heinecke, H. Pabst, and G. Henry, *LIBXSMM: A high performance library for small matrix multiplications.*
[20] P. Hemker, W. Hoffmann, and M. Van Raalte, *Two-level fourier analysis of a multigrid approach for discontinuous Galerkin discretization*, SIAM Journal on Scientific Computing, 25 (2003), pp. 1018–1041.
[21] P. Hemker, W. Hoffmann, and M. Van Raalte, *Fourier two-level analysis for discontinuous Galerkin discretization with linear elements*, Numerical linear algebra with applications, 11 (2004), pp. 473–491.
[22] P. Hemker and M. Van Raalte, *Fourier two-level analysis for higher dimensional discontinuous Galerkin discretisation*, Computing and Visualization in Science, 7 (2004), pp. 159–172.
[23] *Intel Math Kernel Library. reference manual*, 2009.
[24] J. Jeffers, J. Reinders, and A. Sodani, *Intel Xeon Phi Processor High Performance Programming: Knights Landing Edition*, Morgan Kaufmann, 2016.
[25] K. Johannsen, *Multigrid methods for nipg*, ICES Report, (2005), pp. 05–31.
[26] K. Johannsen, *A symmetric smoother for the nonsymmetric interior penalty discontinuous Galerkin discretization*, ICES Report, 5 (2005), p. 23.
[27] G. Karniadakis and S. J. Sherwin, *Spectral/hp Element Methods for CFD*, Numerical mathematics and scientific computation, Oxford University Press, 1999.
[28] J. King, S. Yakovlev, Z. Fu, R. M. Kirby, and S. J. Sherwin, *Exploiting batch processing on streaming architectures to solve 2d elliptic finite element problems: A hybridized discontinuous Galerkin (HDG) case study*, Journal of Scientific Computing, 60 (2014), pp. 457–482.
[29] R. M. Kirby, S. J. Sherwin, and B. Cockburn, *To cg or to hdg: A comparative study*, Journal of Scientific Computing, 51 (2012), pp. 183–212.
[30] M. G. Knepley, K. Rupp, and A. R. Terrel, *Finite element integration with quadrature on the GPU*, CoRR, abs/1607.04245 (2016).

[31] M. Kronbichler and K. Kormann, *A generic interface for parallel cell-based finite element operator application*, Computers & Fluids, 63 (2012), pp. 135–147.
[32] J. D. McCalpin, *STREAM: Sustainable memory bandwidth in high performance computers*. https://www.cs.virginia.edu/stream/. Accessed: 2016-08-30.
[33] A. Napov and Y. Notay, *When does two-grid optimality carry over to the v-cycle?*, Numerical linear algebra with applications, 17 (2010), pp. 273–290.
[34] X. Roca, N. C. Nguyen, and J. Peraire, *Gpu-accelerated sparse matrix-vector product for a hybridizable discontinuous Galerkin method*, in Aerospace Sciences Meetings. American Institute of Aeronautics and Astronautics, 2011, pp. 2011–687.
[35] L. N. Trefethen, *Approximation Theory and Approximation Practice*, Siam, 2013.
[36] C. W. O. U. Trottenberg and A. Schüller, *Multigrid*, Academic Press, 2001.
[37] P. E. Vos, S. J. Sherwin, and R. M. Kirby, *From h to p efficiently: Implementing finite and spectral/hp element methods to achieve optimal performance for low-and high-order discretisations*, Journal of Computational Physics, 229 (2010), pp. 5161–5181.
[38] H. Wang, D. Huybrechs, and S. Vandewalle, *Explicit barycentric weights for polynomial interpolation in the roots or extrema of classical orthogonal polynomials*, Mathematics of Computation, 83 (2014), pp. 2893–2914.
[39] P. Wesseling, *An introduction to multigrid methods*, Pure and applied mathematics, John Wiley & Sons Australia, Limited, 1992.
[40] S. Williams, A. Waterman, and D. Patterson, *Roofline: An insightful visual performance model for multicore architectures*, Commun. ACM, 52 (2009), pp. 65–76.

View publication stats